Building Enterprise Systems with the ZRB Methodology and Toolkit

Version: 0.1.0
Date: March 2026
Author: Harris Wang
Contact: harrisw@athabascau.ca
Repository: https://github.com/hongxueharriswang/zrb_toolkit

---

1. Introduction

Welcome! In this tutorial you will learn how to design and implement a secure, maintainable enterprise information system using the **Zoned Role-Based (ZRB) framework** and its Python toolkit. You will build a simplified **University Course Management System** step by step, and by the end you will be able to apply the same principles to any real-world domain.

The ZRB approach flips traditional access control on its head: instead of adding security later, you model your organisation's structure (departments, teams, projects) as a **zone tree**. Within each zone you define **roles**, assign **operations** (permissions), and let **inheritance** (both within a zone and between zones) reduce redundancy. **Constraints** (separation of duty, time-based, attribute-based) give you fine-grained control.

The **ZRB toolkit** provides:

- A **storage layer** (SQLAlchemy) to persist your configuration.
- An **access control engine** that computes effective permissions and evaluates constraints efficiently.
- **Web framework integrations** (Django and Flask) with simple decorators to protect your views.
- A **command-line interface** to manage configurations.

Let's dive in.

---

2. Prerequisites

- Python 3.9 or higher.
- Basic knowledge of Python and either Django or Flask (we will show both).
- Familiarity with YAML (helpful but not essential).

For the latest development version, clone the repository and install with pip:

```
git clone https://github.com/hongxueharriswang/zrb_toolkit.git
cd zrb_toolkit
pip install .
```

Verify the installation by importing the package:

```
import zrb
print(zrb.__version__)   # should print 0.1.0 or later
```

For Django projects, also install Django:

```
pip install django
```

For Flask projects, install Flask and Flask-Login (or your preferred auth extension):

```
pip install flask flask-login
```

---

3. Understanding ZRB Core Concepts

Before coding, let's get comfortable with the main building blocks.

3.1 Zones

A **zone** represents an organisational unit. Zones form a tree – the root is the entire enterprise, children are subdivisions. For a university:

```
University (root)
├── Faculty of Engineering
│   ├── Computer Science
│   └── Electrical Engineering
└── Faculty of Arts
    ├── History
    └── Philosophy
```

Each zone has an `id` (unique) and a `parent_id` (null for root).

3.2 Operations

An **operation** is a single protected action, e.g., `grade:submit`, `course:enroll`. Operations are identified by a string and belong to an application (like `grading` or `curriculum`).

3.3 Roles

A **role** is a job function within a specific zone. For example, in the Computer Science zone we might have roles: `professor`, `student`, `TA`. Each role can have **base permissions** – a set of operation IDs directly granted.

Roles can have a **parent role** inside the same zone. This creates an **intra-zone hierarchy**: a senior role (e.g., `professor`) inherits all base permissions of its junior roles (e.g., `TA`, `student`). We define this by setting

`parent_role_id` of the senior role to the junior role's ID.

3.4 Gamma Mappings

Sometimes a role in a child zone should inherit from a role in its parent zone. This is **inter-zone inheritance** and is defined by a **gamma mapping**. For example, the Computer Science `professor` role inherits from the Faculty of Engineering `professor` role. A gamma mapping specifies:

- `child_zone_id`, `child_role_id`
- `parent_zone_id`, `parent_role_id`
- optional `weight` (0-1) and `priority` (for conflicts)

3.5 Users and Assignments

A **user** is an individual with an `id` (matching your authentication system). Users are assigned to roles within specific zones via **assignments**.

3.6 Constraints

Constraints are rules that can deny access even if the user has the permission. Types:

- **Separation of duty** (SoD): e.g., a person cannot approve their own purchase order.
- **Temporal**: e.g., grade submission only allowed during exam week.
- **Attribute**: e.g., reorder only if stock < 10.
- **Context**: e.g., test runs allowed only in QA environment.

Constraints are evaluated during access decisions and can access a **context dictionary** (provided by the application) that contains dynamic information like current time, resource owner, or stock level.

3.7 Access Modes

- **Direct (`n_rzbac`)**: only base permissions of the assigned roles are considered. Inheritance is ignored. Use for safety-critical actions.
- **Inferential (`i_rzbac`)**: full effective permissions, including all inheritance. This is the default for most operations.

---

4. Designing the University System

We'll design a system with two faculties and two departments each. For brevity, we focus on the Computer Science department.

Step 1: Zone Tree

```
University (root)
├── Faculty of Engineering (eng)
│   ├── Computer Science (cs)
│   └── Electrical Engineering (ee)
└── Faculty of Arts (arts)
```

```
    ├── History (hist)
    └── Philosophy (phil)
```

Step 2: Operations

We'll need:

- `grade:view` – view own grades
- `grade:submit` – submit final grades (professors only)
- `grade:enter` – enter grades as a TA
- `course:enroll` – students enroll
- `course:view` – view course list
- `course:create` – create a new course (deans only)
- `faculty:report` – view faculty report (deans)
- `schedule:manage` – manage schedule (department heads)
- `prof:assign` – assign professors to courses (department heads)
- `user:manage`, `zone:manage` – admin operations

Step 3: Roles per Zone

- **University root**: `admin` (with `user:manage`, `zone:manage`)
- **Faculty of Engineering**: `dean_eng` (with `course:create`, `faculty:report`), `prof_eng` (with `grade:submit`, `course:view`), `student_eng` (with `grade:view`, `course:enroll`), `ta_eng` (with `grade:enter`)
- **Computer Science**: `head_cs` (with `schedule:manage`, `prof:assign`), `prof_cs` (no base permissions, inherits from `prof_eng` via gamma), `student_cs` (inherits from `student_eng`), `ta_cs` (inherits from `ta_eng`)

Step 4: Intra-zone Hierarchies

In each zone, we want senior roles to inherit from junior ones. For Engineering:

- `dean_eng` inherits from `prof_eng`
- `prof_eng` inherits from `ta_eng`
- `ta_eng` inherits from `student_eng`

We encode this by setting `parent_role_id` of each role to the next junior.

Step 5: Gamma Mappings

We need mappings from each child-zone role to its parent-zone counterpart:

- `(cs, prof_cs)` → `(eng, prof_eng)`
- `(cs, student_cs)` → `(eng, student_eng)`
- `(cs, ta_cs)` → `(eng, ta_eng)`
- Similarly for `ee`, `hist`, `phil`.

Step 6: Constraints

- **SoD**: A student cannot be a TA in the same course (we'll use `cannot_have_role: student` on operation `grade:enter`).
- **Temporal**: Grade submission only allowed between May 1 and June 15, 2025.

Step 7: Users and Assignments

We'll create a few sample users:

- `alice` – professor and head of CS
- `bob` – CS student
- `charlie` – CS TA
- `dave` – Dean of Engineering
- `eve` – Arts professor

Assign them accordingly.

---

5. Writing the ZRB Configuration (YAML)

Create a file named `university_config.yaml`:

```yaml
zones:
  - id: root
    name: University
  - id: eng
    name: Faculty of Engineering
    parent_id: root
  - id: cs
    name: Computer Science
    parent_id: eng
  - id: ee
    name: Electrical Engineering
    parent_id: eng
  - id: arts
    name: Faculty of Arts
    parent_id: root
  - id: hist
    name: History
    parent_id: arts
  - id: phil
    name: Philosophy
    parent_id: arts

operations:
  - id: user:manage
    app_name: admin
    name: Manage users
  - id: zone:manage
    app_name: admin
    name: Manage zones
  - id: course:create
    app_name: curriculum
```

```yaml
      name: Create course
    - id: faculty:report
      app_name: reports
      name: View faculty report
    - id: grade:submit
      app_name: grading
      name: Submit grades
    - id: course:view
      app_name: curriculum
      name: View courses
    - id: grade:view
      app_name: grading
      name: View grades
    - id: course:enroll
      app_name: enrollment
      name: Enroll in course
    - id: grade:enter
      app_name: grading
      name: Enter grades (TA)
    - id: schedule:manage
      app_name: schedule
      name: Manage schedule
    - id: prof:assign
      app_name: faculty
      name: Assign professor

  roles:
    - id: admin
      zone_id: root
      name: Admin
      base_permissions: ["user:manage", "zone:manage"]

    - id: dean_eng
      zone_id: eng
      name: Dean
      base_permissions: ["course:create", "faculty:report"]
      parent_role_id: prof_eng
    - id: prof_eng
      zone_id: eng
      name: Professor
      base_permissions: ["grade:submit", "course:view"]
      parent_role_id: ta_eng
    - id: ta_eng
      zone_id: eng
      name: TA
      base_permissions: ["grade:enter"]
      parent_role_id: student_eng
    - id: student_eng
      zone_id: eng
      name: Student
      base_permissions: ["grade:view", "course:enroll"]

    - id: head_cs
      zone_id: cs
```

```yaml
      name: DeptHead
      base_permissions: ["schedule:manage", "prof:assign"]
      parent_role_id: prof_cs
    - id: prof_cs
      zone_id: cs
      name: Professor
      base_permissions: []
    - id: ta_cs
      zone_id: cs
      name: TA
      base_permissions: []
    - id: student_cs
      zone_id: cs
      name: Student
      base_permissions: []

    # Similar for ee, arts, hist, phil (omitted for brevity)

  gamma_mappings:
    - child_zone_id: cs
      child_role_id: prof_cs
      parent_zone_id: eng
      parent_role_id: prof_eng
      priority: 1
    - child_zone_id: cs
      child_role_id: student_cs
      parent_zone_id: eng
      parent_role_id: student_eng
      priority: 1
    - child_zone_id: cs
      child_role_id: ta_cs
      parent_zone_id: eng
      parent_role_id: ta_eng
      priority: 1
    # ... mappings for ee, hist, phil

  constraints:
    - id: sod_student_ta
      type: separation_of_duty
      target: { operation_id: "grade:enter" }
      condition: { cannot_have_role: "student" }
      is_negative: true
      priority: 10
    - id: temporal_grade_submit
      type: temporal
      target: { operation_id: "grade:submit" }
      condition: { time_range: ["2025-05-01T00:00", "2025-06-15T23:59"] }
      is_negative: false
      priority: 10

  users:
    - id: u1
      username: alice
      email: alice@uni.edu
```

```yaml
    attributes: {}
  - id: u2
    username: bob
    email: bob@uni.edu
    attributes: {}
  - id: u3
    username: charlie
    email: charlie@uni.edu
    attributes: {}
  - id: u4
    username: dave
    email: dave@uni.edu
    attributes: {}
  - id: u5
    username: eve
    email: eve@uni.edu
    attributes: {}

assignments:
  - user_id: u1
    zone_id: cs
    role_id: prof_cs
  - user_id: u1
    zone_id: cs
    role_id: head_cs
  - user_id: u2
    zone_id: cs
    role_id: student_cs
  - user_id: u3
    zone_id: cs
    role_id: ta_cs
  - user_id: u4
    zone_id: eng
    role_id: dean_eng
  - user_id: u5
    zone_id: arts
    role_id: prof_arts
```

---

6. Loading the Configuration

We'll write a small Python script to load this YAML into the ZRB database. Create a file `load_config.py`:

```python
import yaml
from zrb.storage.sqlalchemy import SQLAlchemyStore
from zrb.core.models import User, Zone, Role, Operation, GammaMapping,
Constraint, UserZoneRole
from zrb.core.types import ConstraintType

def load_config(yaml_file, db_url='sqlite:///zrb.db'):
    store = SQLAlchemyStore(db_url)
```

```
    store.create_all()

    with open(yaml_file) as f:
        config = yaml.safe_load(f)

    # Zones
    for z in config['zones']:
        zone = Zone(**z)
        store.create_zone(zone)
        print(f"Zone: {zone.name}")

    # Operations
    for o in config['operations']:
        op = Operation(**o)
        store.create_operation(op)
        print(f"Operation: {op.name}")

    # Roles
    for r in config['roles']:
        role = Role(
            id=r['id'],
            zone_id=r['zone_id'],
            name=r['name'],
            parent_role_id=r.get('parent_role_id'),
            description=r.get('description', ''),
            base_permissions=set(r.get('base_permissions', []))
        )
        store.create_role(role)
        print(f"Role: {role.name} in zone {role.zone_id}")

    # Gamma mappings
    for g in config.get('gamma_mappings', []):
        gamma = GammaMapping(**g)
        store.create_gamma_mapping(gamma)
        print(f"Gamma: {gamma.child_role_id} -> {gamma.parent_role_id}")

    # Constraints
    for c in config.get('constraints', []):
        c['type'] = ConstraintType(c['type'])
        constraint = Constraint(**c)
        store.create_constraint(constraint)
        print(f"Constraint: {constraint.id}")

    # Users
    for u in config.get('users', []):
        user = User(**u)
        store.create_user(user)
        print(f"User: {user.username}")

    # Assignments
    for a in config.get('assignments', []):
        uzr = UserZoneRole(**a)
        store.assign_user_to_role(uzr.user_id, uzr.zone_id, uzr.role_id)
        print(f"Assignment: user {uzr.user_id} -> role {uzr.role_id} in
```

```
zone {uzr.zone_id}")

    print("Configuration loaded successfully.")

if __name__ == '__main__':
    load_config('university_config.yaml')
```

Run it:

```
python load_config.py
```

You should see output confirming each entity was created. The database file `zrb.db` now contains your ZRB configuration.

---

7. Building the Web Application – Django

We'll create a Django project that uses the ZRB database and protects its views with decorators.

7.1 Create Django Project and App

```
django-admin startproject university_project
cd university_project
python manage.py startapp university
```

7.2 Install Required Packages

Make sure you have `zrb-toolkit` installed. Also install `pyyaml` if you plan to load config (already done). For production, use a proper database like PostgreSQL, but we'll stick with SQLite for this tutorial.

7.3 Configure Django Settings (`university_project/settings.py`)

Add the app and middleware. Also define the ZRB database URL.

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

```
    'university',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'university.middleware.ZoneMiddleware',   # custom middleware
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',      # business data
    }
}

# ZRB configuration
ZRB_DATABASE_URL = 'sqlite:///' + str(BASE_DIR / 'zrb.db')
```

7.4 Create the Zone Middleware (`university/middleware.py`)

This middleware determines the current zone from the request's subdomain and attaches it to `request.zone`.

```
from django.conf import settings
from zrb.storage.sqlalchemy import SQLAlchemyStore

# Initialize the ZRB store once
zrb_store = SQLAlchemyStore(settings.ZRB_DATABASE_URL)

class ZoneMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        host = request.get_host().split(':')[0]
        # Map subdomain to zone ID (as defined in config)
        subdomain = host.split('.')[0] if '.' in host else None
        zone_map = {
            'cs': 'cs',
            'ee': 'ee',
            'eng': 'eng',
            'hist': 'hist',
            'phil': 'phil',
            'arts': 'arts',
            None: 'root',
```

```
        }
        zone_id = zone_map.get(subdomain, 'root')
        request.zone = zrb_store.get_zone(zone_id)
        return self.get_response(request)
```

7.5 Create the Access Engine Singleton

We need a globally accessible engine for the decorators. We'll create a separate module `university/zrb_setup.py`:

```
from django.conf import settings
from zrb.storage.sqlalchemy import SQLAlchemyStore
from zrb.engine.access import AccessEngine

store = SQLAlchemyStore(settings.ZRB_DATABASE_URL)
engine = AccessEngine(store)

# Also make the store and engine available to decorators by setting module-
level variables
# The decorators in zrb.web.django expect a module-level 'engine' and
'store'.
# We can import and set them.
import zrb.web.django
zrb.web.django.engine = engine
zrb.web.django.store = store
```

Now import this module in `apps.py` or in the middleware's `__init__` to ensure it runs once.

In `university/apps.py`:

```
from django.apps import AppConfig

class UniversityConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'university'

    def ready(self):
        import university.zrb_setup  # noqa
```

7.6 Define Business Models (`university/models.py`)

We'll need models for courses, enrollments, and grades.

```
from django.db import models
from django.contrib.auth.models import User

class Course(models.Model):
```

```
    code = models.CharField(max_length=10, unique=True)
    name = models.CharField(max_length=100)
    credits = models.IntegerField()
    zone_id = models.CharField(max_length=50)  # e.g., 'cs', 'eng'

    def __str__(self):
        return f"{self.code}: {self.name}"

class Enrollment(models.Model):
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('student', 'course')

class Grade(models.Model):
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    grade = models.CharField(max_length=2)
    submitted_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, related_name='grades_submitted')
    submitted_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('student', 'course')
```

Run migrations:

```
python manage.py makemigrations university
python manage.py migrate
```

7.7 Create Protected Views (`university/views.py`)

We'll use the decorators from `zrb.web.django`.

```
import json
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse, HttpResponseForbidden
from django.views.decorators.csrf import csrf_exempt
from zrb.web.django import i_rzbac, n_rzbac
from .models import Course, Enrollment, Grade
from .zrb_setup import store, engine  # we have engine, but decorators use
global

@login_required
@i_rzbac(operation='grade:view')
def view_grades(request):
    grades =
```

```
Grade.objects.filter(student=request.user).values('course__code', 'grade')
    return JsonResponse(list(grades), safe=False)

@login_required
@csrf_exempt
@n_rzbac(operation='grade:submit')
def submit_grade(request):
    if request.method != 'POST':
        return JsonResponse({'error': 'Method not allowed'}, status=405)
    data = json.loads(request.body)
    grade = Grade.objects.create(
        student_id=data['student_id'],
        course_id=data['course_id'],
        grade=data['grade'],
        submitted_by=request.user
    )
    return JsonResponse({'status': 'submitted', 'id': grade.id})

@login_required
@csrf_exempt
@i_rzbac(operation='course:enroll')
def enroll(request):
    if request.method != 'POST':
        return JsonResponse({'error': 'Method not allowed'}, status=405)
    data = json.loads(request.body)
    course = Course.objects.get(id=data['course_id'])
    enrollment, created =
Enrollment.objects.get_or_create(student=request.user, course=course)
    return JsonResponse({'status': 'enrolled' if created else 'already
enrolled'})

@login_required
@i_rzbac(operation='course:view')
def list_courses(request):
    zone_id = request.zone.id if request.zone else 'root'
    courses = Course.objects.filter(zone_id=zone_id).values('id', 'code',
'name')
    return JsonResponse(list(courses), safe=False)

# Admin views (require zone:manage or user:manage)
@login_required
@i_rzbac(operation='zone:manage')
def manage_zones(request):
    return JsonResponse({'message': 'Zone management endpoint'})

@login_required
@i_rzbac(operation='user:manage')
def manage_users(request):
    return JsonResponse({'message': 'User management endpoint'})
```

7.8 Wire Up URLs (`university/urls.py`)

```
from django.urls import path
from . import views

urlpatterns = [
    path('grades/', views.view_grades, name='view_grades'),
    path('grades/submit/', views.submit_grade, name='submit_grade'),
    path('enroll/', views.enroll, name='enroll'),
    path('courses/', views.list_courses, name='list_courses'),
    path('admin/zones/', views.manage_zones, name='manage_zones'),
    path('admin/users/', views.manage_users, name='manage_users'),
]
```

Include in project `urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('university.urls')),
]
```

7.9 Add Login (Simplified)

For testing, you can use Django's admin login. In `settings.py`, set:

```
LOGIN_URL = '/admin/login/'
```

Create a superuser:

```
python manage.py createsuperuser
```

Now you can log in as that user. However, the ZRB users must match Django's user IDs. In our config, user IDs are `u1`, `u2`, etc., but Django's `User` model uses integer primary keys. You have two options:

- Modify the ZRB config to use numeric IDs matching Django's user IDs.
- Or, create Django users with `id` matching the string `u1`? That's not possible because Django expects integer. Better to adapt: when loading config, also create Django users with matching IDs. In our earlier load script, we didn't create Django users. For simplicity in this tutorial, we assume you manually create Django users with IDs 1,2,3,... and then update the ZRB config to use those numeric IDs (as strings). Or you can modify the decorators to look up ZRB user by username instead of ID. We'll keep it simple and assume you adapt the config.

Alternatively, you can bypass Django's auth and use a custom authentication that uses ZRB users directly, but that's beyond this tutorial.

For now, we'll proceed assuming the IDs match.

7.10 Run the Development Server

```
python manage.py runserver
```

Test by accessing `http://cs.university.local:8000/api/courses/` – you'll need to add `cs.university.local` to your hosts file pointing to 127.0.0.1. Also log in first.

---

8. Building the Web Application – Flask

If you prefer Flask, here is the equivalent.

8.1 Project Structure

```
university_flask/
├── app.py
├── zrb_config.yaml
├── load_config.py   (same as before)
└── requirements.txt
```

8.2 Flask Application (`app.py`)

```
import os
import yaml
from flask import Flask, request, jsonify, abort
from flask_login import LoginManager, UserMixin, login_user, current_user,
login_required
from zrb.storage.sqlalchemy import SQLAlchemyStore
from zrb.engine.access import AccessEngine
from zrb.web.flask import ZRBFlask
from zrb.core.models import User as ZRBUser

app = Flask(__name__)
app.secret_key = 'dev-secret-key'

# Database
db_url = 'sqlite:///zrb.db'
store = SQLAlchemyStore(db_url)
engine = AccessEngine(store)

# ZRB Flask extension
zrb = ZRBFlask(app, engine)
```

```python
# Flask-Login
login_manager = LoginManager()
login_manager.init_app(app)

# Dummy user class
class User(UserMixin):
    def __init__(self, id, username):
        self.id = id
        self.username = username

# User loader - in real app, fetch from your DB
users_db = {
    '1': {'username': 'alice', 'zrb_id': 'u1'},
    '2': {'username': 'bob', 'zrb_id': 'u2'},
    # ... etc.
}

@login_manager.user_loader
def load_user(user_id):
    if user_id in users_db:
        return User(user_id, users_db[user_id]['username'])
    return None

@app.route('/login/<user_id>')
def login(user_id):
    if user_id not in users_db:
        abort(404)
    user = User(user_id, users_db[user_id]['username'])
    login_user(user)
    return f"Logged in as {user.username}"

# Zone resolver (called by ZRBFlask before_request)
def zone_from_request():
    host = request.host.split(':')[0]
    subdomain = host.split('.')[0] if '.' in host else None
    zone_map = {
        'cs': 'cs',
        'eng': 'eng',
        None: 'root',
    }
    zone_id = zone_map.get(subdomain, 'root')
    return store.get_zone(zone_id)

zrb.zone_resolver = zone_from_request

# Protected endpoints
@app.route('/api/grades')
@login_required
@zrb.i_rzbac(operation='grade:view')
def view_grades():
    # In a real app, fetch grades from DB
    return jsonify({'grades': []})
```

```
@app.route('/api/grades/submit', methods=['POST'])
@login_required
@zrb.n_rzbac(operation='grade:submit')
def submit_grade():
    return jsonify({'status': 'submitted'})

@app.route('/api/enroll', methods=['POST'])
@login_required
@zrb.i_rzbac(operation='course:enroll')
def enroll():
    return jsonify({'status': 'enrolled'})

@app.route('/api/courses')
@login_required
@zrb.i_rzbac(operation='course:view')
def list_courses():
    return jsonify({'courses': []})

if __name__ == '__main__':
    app.run(debug=True)
```

8.3 Run Flask

```
pip install flask flask-login pyyaml zrb-toolkit
python load_config.py university_config.yaml
python app.py
```

Access via `http://cs.university.local:5000/api/courses` after setting hosts file.

---

9. Handling Constraints with Context

The real power of ZRB comes from constraints. To evaluate a temporal constraint, the engine needs the current time. The decorators do not automatically pass context; you have two choices:

Option A: Direct Engine Call

Skip the decorator and call `engine.decide()` manually, passing a context dictionary.

```
from .zrb_setup import store, engine

def submit_grade(request):
    data = json.loads(request.body)
    context = {'current_time': datetime.now().isoformat()}
    user = store.get_user(str(request.user.id))
    op = store.get_operation('grade:submit')
    zone = request.zone
    if not engine.decide(user, op, zone, mode='inferential',
context=context):
```

```
        return HttpResponseForbidden()
    # ... proceed
```

Option B: Custom Decorator with Context

Create a decorator that accepts a callable to build context.

```
def i_rzbac_with_context(operation=None, context_func=None):
    def decorator(view):
        @wraps(view)
        def _wrapped(request, *args, **kwargs):
            context = context_func(request) if context_func else {}
            user = store.get_user(str(request.user.id))
            op = store.get_operation(operation or
request.resolver_match.url_name)
            zone = request.zone
            if not engine.decide(user, op, zone, mode='inferential',
context=context):
                return HttpResponseForbidden()
            return view(request, *args, **kwargs)
        return _wrapped
    return decorator
```

Then use:

```
@i_rzbac_with_context(operation='grade:submit', context_func=lambda r:
{'current_time': datetime.now().isoformat()})
def submit_grade(request):
    ...
```

For Flask, the pattern is similar.

---

10. Testing and Simulation

You can write unit tests using the engine directly.

```
import pytest
from zrb.storage.sqlalchemy import SQLAlchemyStore
from zrb.engine.access import AccessEngine

@pytest.fixture
def store():
    # Use an in-memory SQLite database for testing
    store = SQLAlchemyStore('sqlite:///:memory:')
    store.create_all()
    # Load test configuration (you can load from YAML)
```

```
    yield store

def test_professor_can_submit_grades(store):
    # Assume we have a professor user and grade:submit operation
    engine = AccessEngine(store)
    user = store.get_user('u1')
    zone = store.get_zone('cs')
    op = store.get_operation('grade:submit')
    assert engine.decide(user, op, zone) is True
```

You can also simulate a large number of requests to measure performance.

---

11. Deployment Considerations

- Use a production database (PostgreSQL) for the ZRB store.
- Set up proper caching (Redis) if needed.
- Run your Django/Flask app with Gunicorn behind Nginx.
- Configure subdomains correctly in DNS and in your middleware.
- Regularly back up the ZRB configuration (YAML) and database.

---

12. Conclusion

Congratulations! You have built a complete, secure enterprise system using the ZRB methodology and toolkit. You have learned how to:

- Model an organisation as a zone tree.
- Define roles, operations, and inheritance.
- Enforce constraints.
- Integrate with Django and Flask.
- Test and simulate access decisions.

The same approach scales to any domain – hospital, banking, e-commerce, project management. The ZRB toolkit gives you a solid foundation to build systems that are **secure by design** and easy to maintain.

For further reading, consult the ZRB Toolkit API Reference and explore the example systems provided in the repository.

Happy coding!